Part 1 – Pattern discovery: Member 3 prompt and result

Course 23CSE455 Design Patterns, Team A15
Member: Sreyas A (AM.AI.U4AID23022)
Scope: persistence layer (persistence/)
Model: Claude Sonnet 5.5 (configured id claude-sonnet-5-5), 2026-10-11
Attached file: `input_source_numbered.txt` (numbered Java source of the baseline application, 20 KB; it is in `Part1/llm/`).

Prompt

```
[Member 3 prompt - Part 1]
SCOPE: persistence layer = persistence/dao/ (DAO, AccountDAO, UserDAO), persistence/model/
(Account, AccountType, User) and src/main/resources/application.properties.
You may name classes outside the scope when they are collaborators, but only report candidates whose
main participants are in this scope.

You are assisting with a university Design Patterns case study (course 23CSE455).

INPUT: the numbered Java source of a Quarkus bank REST API (baseline version), supplied as
input_source_numbered.txt. Every source line starts with its real line number followed by '|'.

TASK (Part 1, pattern discovery) - restricted to the scope given below:
1. Find every pattern candidate in your scope: GoF patterns, GRASP patterns, enterprise/architecture
   patterns, and framework idioms. Give the exact recognised pattern name and its category.
2. For each candidate give a meaningful instance name (one collaboration, e.g. "Book Return Notification",
   not just "Strategy").
3. Map every pattern role to the ACTUAL class / interface / method in the code. Do not invent classes.
   If a role is combined or absent, say "not separately declared".
4. Give a file path and line range for every participant, using this exact row format:
       ROLE | SYMBOL | path:start-end
   where path is relative to src/main/java/br/com/michaelcruz/ (or src/main/resources/).
5. Give concise evidence from the code (relationships, calls, delegation, behaviour).
   A class name or an annotation alone is NOT sufficient evidence.
6. Give a verdict for each candidate: CONFIRMED, REJECTED or UNCERTAIN, with a reason.
   Two concrete classes in one collaboration are ONE instance.
7. Finish with the counts for your scope: confirmed pattern types, confirmed instances,
   concrete participant classes, rejected candidates, uncertain candidates.
Do not use any knowledge about this repository other than the supplied code.
```

Result

```
# Member 3 - raw model output (scope: persistence/dao, persistence/model, application.properties)

## Candidate 1 - "Account and User Persistence DAO" - Data Access Object (Core J2EE) - CONFIRMED
Problem solved: keep persistence calls out of the business layer behind a small, entity-oriented API.

DataAccessObject | DAO<T> | persistence/dao/DAO.java:6-11
ConcreteDAO | AccountDAO | persistence/dao/AccountDAO.java:13-45
ConcreteDAO | UserDAO | persistence/dao/UserDAO.java:13-38
DataSource | EntityManager (jakarta.persistence, injected) | persistence/dao/AccountDAO.java:16-17
DataSource | EntityManager (jakarta.persistence, injected) | persistence/dao/UserDAO.java:16-17
TransferObject | Account | persistence/model/Account.java:5-44
TransferObject | User | persistence/model/User.java:5-56
BusinessObject | AccountService | service/AccountService.java:18-85
BusinessObject | UserService | service/UserService.java:15-46
Operation | AccountDAO.findById | persistence/dao/AccountDAO.java:19-28
Operation | AccountDAO.save | persistence/dao/AccountDAO.java:30-33
Operation | AccountDAO.update | persistence/dao/AccountDAO.java:35-39
Operation | AccountDAO.getAll | persistence/dao/AccountDAO.java:41-44
Operation | UserDAO.findById | persistence/dao/UserDAO.java:19-26
Operation | UserDAO.save | persistence/dao/UserDAO.java:28-32
Operation | UserDAO.getAll | persistence/dao/UserDAO.java:34-37

Evidence: AccountDAO encapsulates entityManager.find (line 21), persist (line 32), merge (line 38) and a JPQL
query (line 43); UserDAO wraps find (22) and persist (30). Services call only these DAO methods and never import
EntityManager. The interface DAO<T> declares only findById and getAll (lines 8-9); save and update exist only on the
concrete classes, so the services depend on the concrete DAO classes rather than on the interface (a weakened
abstraction, but the pattern is still present because two entity-specific DAOs share one contract).
Defects noticed while reading: UserDAO.findById uses Optional.of(entityManager.find(..)) (line 22), which throws
NullPointerException for an unknown id; the catch of NotFoundException (line 23) can never trigger because
```

EntityManager.find does not throw it; UserDAO.getAll returns null (line 36).

Rejected / uncertain
- Data Mapper (JPA / Hibernate EntityManager mapping Account and User to tables) - UNCERTAIN: the mapper is the persistence provider (dependency); the code in scope only supplies mapping annotations (@Entity, @Id, @OneToOne), so no application-written mapper participants can be located.
- Interceptor / declarative transaction via @Transactional (AccountDAO lines 30 and 35, UserDAO line 28) - UNCERTAIN: only the annotation is visible; the interceptor is in a dependency and nothing in the supplied code proves its behaviour.
- Active Record - REJECTED: Account (5-44) and User (5-56) contain no persistence behaviour.
- Repository as a separate instance - REJECTED: same collaboration as the DAO above under a different name; the classes are named DAO and expose save/update rather than collection-like add/remove.
- Null Object for the Optional returned by findById - REJECTED: Optional.of(null) throws; no neutral object exists.

Counts for my scope
confirmed pattern types: 1; confirmed instances: 1; concrete participant classes: 2 (AccountDAO, UserDAO); rejected: 3; uncertain: 2.
application.properties (lines 1-6) only configures MySQL and hibernate-orm.database.generation=update; it holds no pattern participants.